

CS23602

COMPILER DESIGN

L	T	P	C
3	0	2	4

UNIT – I FRONT END OF COMPILERS

10L, 10P

Language Processors – Structure of a Compiler – Lexical Analysis: Role of Lexical Analyzer – Specification of Tokens – Recognition of Tokens. Syntax Analysis: Introduction – Context Free Grammars – Top Down Parsing – Bottom-Up Parsing – SLR, CLR, LALR Parsers – Using Ambiguous Grammars.

PRACTICALS:

1. Programs using LEX for tokenization.
2. Implementation of error recovery procedures using LEX.
3. Programs using YACC for parsing.
4. Programs for validating C-like constructs using YACC.

UNIT – II INTERMEDIATE CODE GENERATION

10L, 10P

Syntax Directed Definitions – Three Address Code – Types and Declarations – Translations of Expressions – Type Checking – Control Flow – Backpatching – switch statements – Intermediate code for procedures.

PRACTICALS:

1. Implementation of three-address code generation for a programming language like C.
2. Three-address code generation for Switch-case statements.
3. Three-address code generation for arrays.

UNIT – III RUNTIME ENVIRONMENTS

8L

Storage Organization – Stack Allocation Space – Access to Nonlocal Data on the Stack – Heap Management – Introduction to Garbage Collection – Introduction to Trace Based Collection – Short Pause Garbage Collection – Parallel and Concurrent Garbage Collection.

UNIT – IV CODE OPTIMIZATION

8L

Basic Blocks and Flow Graphs – Optimization of Basic Blocks – Peephole Optimization – Principal Sources of Optimization – Introduction to Data Flow Analysis – Partial Redundancy Elimination – Loops in Flow Graphs,

UNIT – V CODE GENERATION

9L, 10P

Issues in the Design of a Code generator – Target Language – Address of the target code – Simple Code Generator – Register Allocation and Code Generation – Instruction Selection by Tree Rewriting – Optimal Code Generation for Expressions – Dynamic Programming Code Generation,

PRACTICALS:

1. Parameter passing techniques for a programming language like C.
2. Symbol table implementation.

3. Implementation of Register allocation using Graph colouring.
4. Simple code generator for a programming language like C.
5. Integrating all the implemented features for a programming language like C.

TOTAL = 75 PERIODS

COURSE OUTCOMES:

Upon completion of the course, the students will be able to

1. Explain the role of various phases of compilation
2. Develop scanner and parser from the given language specification
3. Generate the intermediate representation of programs
4. Produce the object code using the runtime environment
5. Transform given code into an optimized code by applying various optimization techniques

REFERENCES:

1. Alfred Aho, Monica S Lam, Ravi Sethi, Jeffrey D Ullman, "Compilers Principles, Techniques and Tools", Pearson Education, Asia 2014.
2. Andrew W Appel, Modern Compiler Implementation in ML, Cambridge University Press, December 1997.
3. Kenneth C. Loudon, Compiler Construction: Principles and Practice, Cengage Learning, 1st Edition, 1997.
4. Steven. S. Muchnick, Advanced Compiler Design and Implementation, Morgan Kaufman Publishers, First Edition, 1997.
5. Randy Allen and Ken Kennedy, Optimizing Compilers for Modern Architectures: A Dependence based Approach, Morgan Kaufman, First Edition, 2001.
6. Y. N. Srikant, Priti Shankar, The Compiler Design Handbook – Optimizations and Machine Code Generation, CRC Press, Second Edition, 2007.
7. John E Hopcroft and Jeffery D Ullman, "Introduction to Automata Theory, Languages and Computations", Narosa Publishing House, 2002.

CO-PO Mapping

	PO1	PO2	PO3	PO4	PO5	PO6	PO7	PO8	PO9	PO10	PO11	PO12	PSO1	PSO2	PSO3
CO1	1	1	2	2	3	1	1	-	-	-	-	1	3	2	1
CO2	1	2	2	2	3	1	1	-	-	-	1	1	3	2	1
CO3	1	2	2	2	3	1	1	-	-	-	1	1	3	2	1
CO4	1	2	2	2	3	1	1	-	1	-	1	1	3	2	1
CO5	3	3	2	3	3	1	1	-	1	-	1	1	3	2	1

1 - low, 2 - medium, 3 - high, '-' - no correlation